

It is imperative to use the provided squared sheets for the simplified diagrams

An unjustified answer will be considered as false

The aim of this exercise is to study the impact of the data cache on the execution performance of an application. Consider an array of integers and a sort function using the "bubble sort" algorithm to sort the elements of this array in ascending order. The **PushDownMax** function places the largest element at the end of the array by swapping the array cells.

```
void PushDownMax (int *tab, int size)
{
    int i    = 0;
    int tmp  = 0;
    for (i=1 ; i<size ; i++)
    {
        if (tab [i] < tab [i-1])
        {
            tmp      = tab [i ];
            tab [i ] = tab [i-1];
            tab [i-1] = tmp      ;
        }
    }
    return ;
}
```

A compiler for the **Mips** implementation, the 5-stage pipeline of the Mips32 architecture, has generated the following code for the main loop of this function.

At the loop entry, register :

- R4 contains the address of the 2nd element of the array
- R5 contains the end address of the array (the address immediately following the last cell of the array)

```
_For :      Lw      r6 ,  0 (r4 )      ; tab [i ]
           Lw      r7 , -4 (r4 )      ; tab [i-1]
           Slt     r8 , r6 , r7       ; tab [i ] < tab [i-1]
           Beq     r8 , r0 , _Endif
           Nop
           Sw      r7 ,  0 (r4 )
           Sw      r6 , -4 (r4 )
_Endif :    Addiu   r4 , r4 ,  4
           Sltu    r9 , r4 , r5
           Bne     r9 , r0 , _For
           Nop
```

First, the memory system is assumed to be perfect (a load or store takes one cycle).

For questions 1, 2, 3, 4 and 5, we assume that the branch fails 90% of the time.

- 1- On average, how many cycles are required to execute one iteration of the loop on the **Mips** implementation ? You are not asked to draw a diagram, but to clearly indicate stall cycles and data dependencies in the code. Calculate the CPI-useful.
- 2- Analyze the loop code using a simplified diagram on the two-pipeline superscalar **SS2** realization studied in class in the case where the branch fails and in the case where it succeeds. It is assumed that the address at the start of the loop is placed on an aligned address (a multiple of 8) and that the instruction buffer is empty at the start of the loop.
- 3- Reorder the loop to avoid lost cycles for **SS2** realization.
- 4- Unroll the loop once (processing 2 elements at each iteration) and optimize for **Mips** realization.
- 5- Optimize the loop using the "software pipeline" technique to avoid lost cycles for **Mips** realization.

In the rest of the exercise, we consider the initial assembly code and the **Mips** implementation.

The processor is connected to a separate data cache and instruction cache. The caches are connected to memory via a bus. The bus transfers 4 bytes per cycle. The instruction cache is assumed to be perfect (Miss rate = 0%).

The data cache has a size of 2 Kbytes. It's a direct-mapping, write-through cache with a single-place write buffer. A cache block contains 64 bytes.

In the event of a hit, it takes 1 cycle to access the cache. On average, 8 cycles are required between the request for bus access and the authorization to use the bus. In the event of a miss, it takes 1 cycle to detect the miss, then 24 cycles to load the missing block ($8 + 1 \times 64 / 4$; 8 cycles to get the bus and 16 cycles for the transfer on the bus). After loading the block, it takes another 1 cycle to update the cache and 1 more cycle to respond to the processor.

For a write, if there's a place in the buffer, it takes 1 cycle to register the write request in the buffer. It then takes 8 cycles to obtain the access to the bus. Then, 1 cycle to transmit the request to the memory. The write request is removed from the buffer in the same cycle.

Assume that the **tab** array contains 1 K integers (size = 1024; 1 integer is represented by 1 word). The array contains the numbers 1024 to 1 in descending order (**tab[i] = 1024-i**). The address of the array is a multiple of its size (4096).

- 6- How many Misses occur during the execution of the loop ? Calculate the Miss rate.
- 7- Calculate the increase of the average number of cycles due to loads for one iteration.
- 8- Calculate the increase of the average number of cycles due to stores for one iteration. Deduce the average number of cycles per iteration, taking into account the impact of loads and stores.
- 9- It is now assumed that the write buffer has 2 places. Calculate the increase of the average number of cycles due to stores. Deduce the average number of cycles per iteration, taking into account the effect of loads and stores.

We now assume that the cache adopts the Write Back strategy.

- 10- What is the cost of the Miss (the number of stall cycles seen by the processor) (distinguish 2 cases).
- 11- Calculate the number of Misses during the execution of the loop (distinguish between two cases). Calculate the Miss rate (distinguish between 2 cases).
- 12- Calculate the average number of cycles per iteration.